CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 19: PCA and Explained Variance Analysis

Problem Statement: A face-recognition attendance system stores thousands of pixels per face, making matching slow. The team needs fewer features without losing identity information. The Olivetti faces dataset (400 images, 64×64 pixels) or the handwritten digits dataset. The objective is to reduce dimensionality with PCA and choose components using explained variance. Use NumPy, Scikit-learn,Matplotlib.

Tasks:

1. Implement PCA from scratch using the covariance matrix and eigen-decomposition.

2. Verify the result against Scikit-learn's PCA.

3. Plot the scree plot and cumulative explained variance.

4. Find the number of components that retain 95% of variance.

5. Visualise the top eigenfaces and reconstructed faces with 10, 50 and 100 components.

FORMULA:

1. Mean Calculation

Mean = (Sum of all values) / Number of values

2. Mean Centering

X_centered = X - Mean(X)

3. Covariance Matrix

Covariance = (X_centered.T × X_centered) / (n - 1)

4. Eigenvalue Equation

$A \times v = \lambda \times v$

5. Explained Variance Ratio

Explained Variance Ratio = Eigenvalue / Sum of All Eigenvalues

6. Cumulative Explained Variance

Cumulative Variance = Sum of Explained Variance Ratios

7. Number of Components for 95% Variance

Cumulative Variance ≥ 0.95

8. PCA Transformation

X_reduced = X_centered × Eigenvectors

9. Face Reconstruction

X_reconstructed = X_reduced × Eigenvectors.T + Mean(X)

10. Reconstruction Error

MSE = Sum((Original - Reconstructed)²) / Number of Elements

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import fetch_olivetti_faces

from sklearn.decomposition import PCA
```

```
from sklearn.preprocessing import StandardScaler

import time

# 1. Load Olivetti faces dataset

data = fetch_olivetti_faces()

X = data.data

images = data.images

print("Dataset shape:", X.shape)

print("Image shape:", images.shape)

# 2. Center the data

mean_face = np.mean(X, axis=0)

X_centered = X - mean_face

# 3. PCA FROM SCRATCH using eigen-decomposition

# Use the smaller sample-space covariance matrix for efficiency.

# This is mathematically equivalent to pixel-space covariance PCA.

start = time.time()



cov_matrix = np.dot(X_centered, X_centered.T) / (X.shape[0] - 1)

eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

# Sort eigenvalues in descending order

idx = np.argsort(eigenvalues)[::-1]
```

```
eigenvalues = eigenvalues[idx]

eigenvectors = eigenvectors[:, idx]

# Convert eigenvectors into eigenfaces

eigenfaces = np.dot(X_centered.T, eigenvectors)

# Normalize eigenfaces

norms = np.linalg.norm(eigenfaces, axis=0)

eigenfaces = eigenfaces / np.maximum(norms, 1e-12)

# Remove numerical negative eigenvalues

eigenvalues = np.maximum(eigenvalues, 0)

explained_variance = eigenvalues / np.sum(eigenvalues)

cumulative_variance = np.cumsum(explained_variance)

print("\nPCA from scratch completed.")

print("Time taken:", round(time.time() - start, 2), "seconds")

# 4. Verify with Scikit-learn PCA

sk_pca = PCA(n_components=100, svd_solver="randomized",

        random_state=42)

X_sklearn = sk_pca.fit_transform(X)

print("\nScikit-learn PCA completed.")

print("First 5 explained variance ratios:")

print("Scratch:", np.round(explained_variance[:5], 4))
```

```
print("Sklearn:", np.round(sk_pca.explained_variance_ratio_[:5], 4))

# Compare cumulative explained variance

print("Scratch cumulative variance (100 components):",

    round(cumulative_variance[99], 4))

print("Sklearn cumulative variance (100 components):",

    round(np.sum(sk_pca.explained_variance_ratio_), 4))

# 5. Scree plot

plt.figure(figsize=(8, 4))

plt.plot(np.arange(1, len(eigenvalues) + 1),

      explained_variance, marker=".")

plt.title("Scree Plot")

plt.xlabel("Principal Component")

plt.ylabel("Explained Variance Ratio")

plt.grid(True)

plt.show()


# 6. Cumulative explained variance

plt.figure(figsize=(8, 4))

plt.plot(np.arange(1, len(cumulative_variance) + 1),

      cumulative_variance)
```

```
plt.axhline(y=0.95, linestyle="--", label="95% variance")
plt.xlabel("Number of Components")
plt.ylabel("Cumulative Explained Variance")
plt.title("Cumulative Explained Variance")
plt.grid(True)
plt.legend()
plt.show()
# 7. Find components needed for 95% variance
n_95 = np.argmax(cumulative_variance >= 0.95) + 1
print("\nComponents needed to retain 95% variance:", n_95)
# 8. Visualize the top 10 eigenfaces
fig, axes = plt.subplots(2, 5, figsize=(12, 5))
for i, ax in enumerate(axes.flat):
    ax.imshow(eigenfaces[:, i].reshape(64, 64), cmap="gray")
    ax.set_title(f"Eigenface {i + 1}")
    ax.axis("off")
plt.suptitle("Top 10 Eigenfaces")
plt.tight_layout()
plt.show()
# 9. Reconstruct a face using 10, 50 and 100 components
```

```
face_index = 0
original = X[face_index]
centered_face = original - mean_face
fig, axes = plt.subplots(1, 4, figsize=(14, 4))
axes[0].imshow(original.reshape(64, 64), cmap="gray")
axes[0].set_title("Original Face")
axes[0].axis("off")
for ax, k in zip(axes[1:], [10, 50, 100]):
    components = eigenfaces[:, :k]
    weights = np.dot(centered_face, components)
    reconstructed = np.dot(weights, components.T) + mean_face
    ax.imshow(reconstructed.reshape(64, 64), cmap="gray")
    ax.set_title(f"{k} Components")
    ax.axis("off")
plt.suptitle("Face Reconstruction Using PCA")
plt.tight_layout()
plt.show()
# 10. Reconstruction error
print("\nReconstruction Mean Squared Error:")
for k in [10, 50, 100]:
```

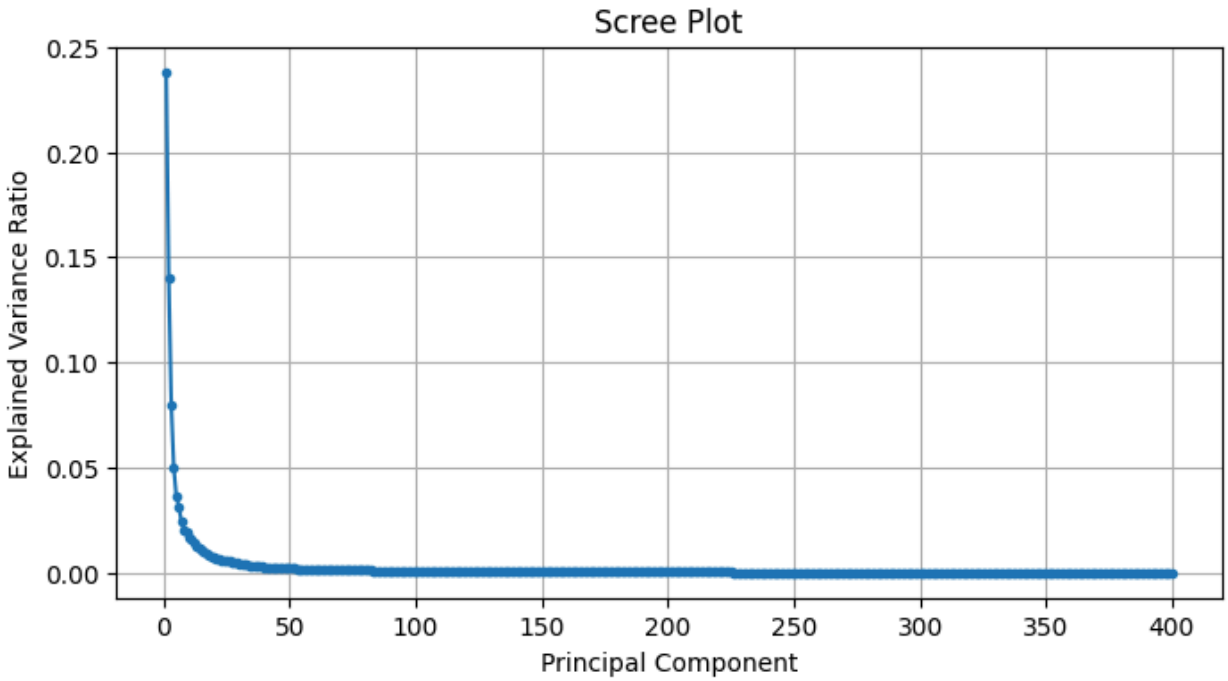
```
    components = eigenfaces[:, :k]
    weights = np.dot(X_centered, components)
    reconstructed = np.dot(weights, components.T) + mean_face
    mse = np.mean((X - reconstructed) ** 2)
    print(f"{k} components: {mse:.6f}")
```

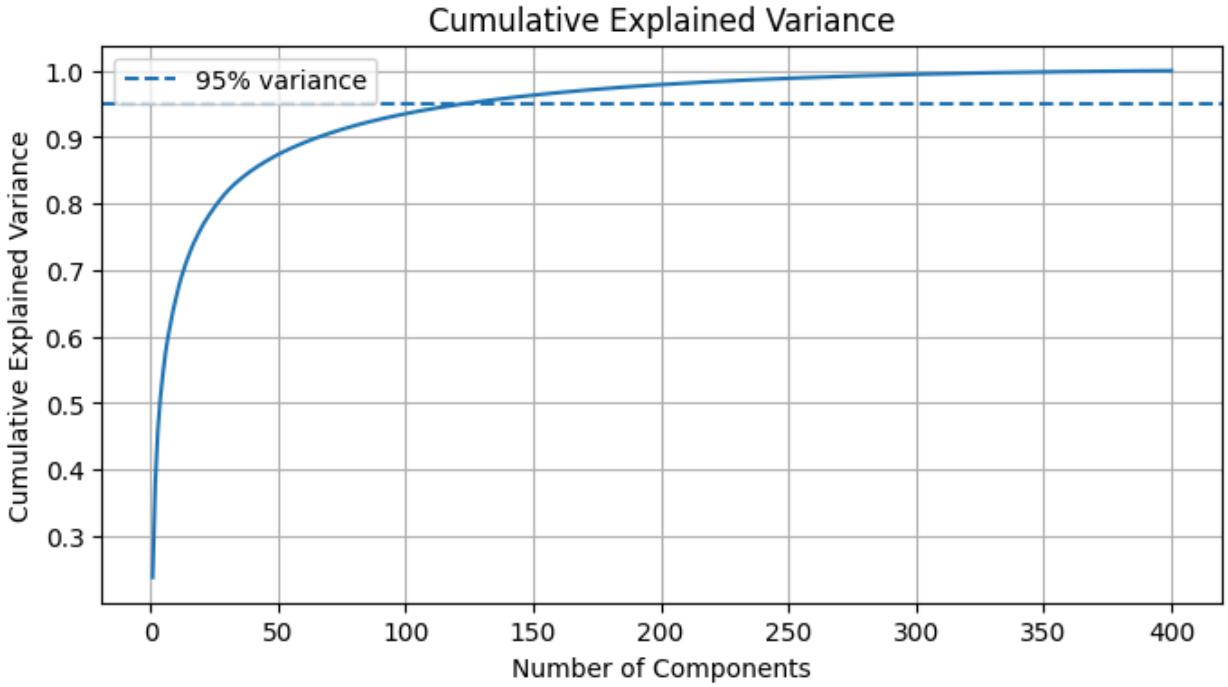
OUTPUT:

```
downloading Olivetti faces from https://ndownloader.figshare.com/files/5976027 to /root/scikit_learn_data
Dataset shape: (400, 4096)
Image shape: (400, 64, 64)

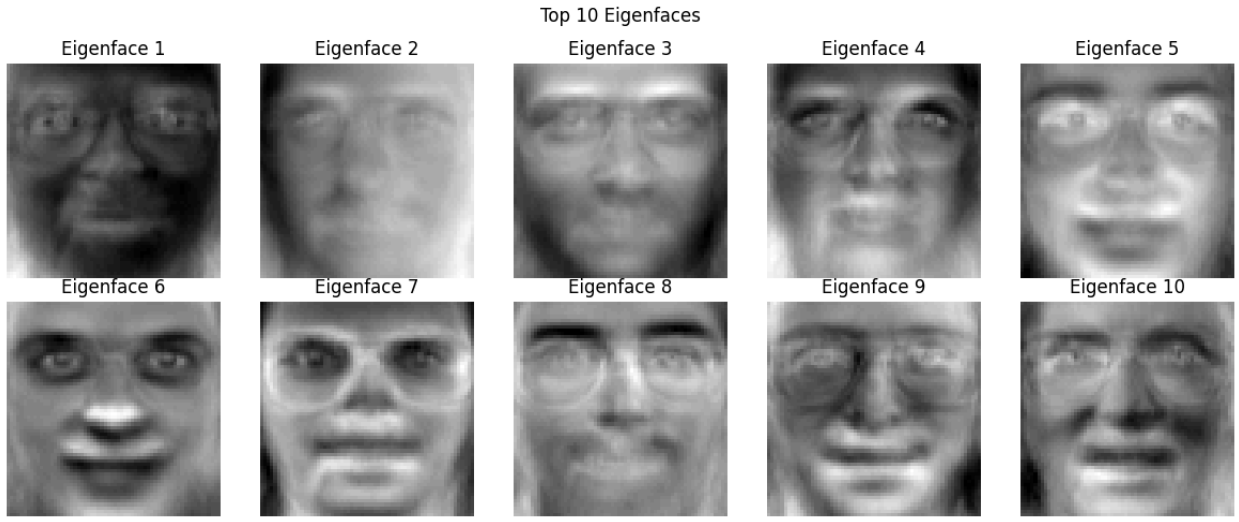
PCA from scratch completed.
Time taken: 0.08 seconds

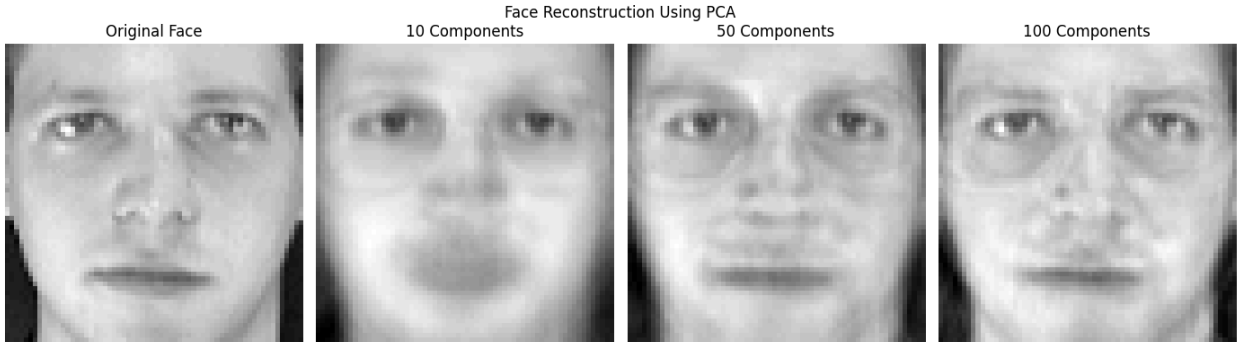
Scikit-learn PCA completed.
First 5 explained variance ratios:
Scratch: [0.2381 0.1399 0.0797 0.05   0.0361]
Sklearn: [0.2381 0.1399 0.0797 0.05   0.0361]
Scratch cumulative variance (100 components): 0.9354
Sklearn cumulative variance (100 components): 0.935
```





Components needed to retain 95% variance: 123





Reconstruction Mean Squared Error:

10 components: 0.006622

50 components: 0.002431

100 components: 0.001246